

PRACTICE ASSESSMENT 1

(SQL DATA TYPES)

Part 1: Simple Table Creation

Exercise 1: Student Information Table

Create a table called students to store student information.

What you need to store:

- Student ID (whole numbers, automatically increases)
- Student name (different length names like "Ali" or "Muhammad Abdullah")
- Roll number (always 6 characters like "CS0001")
- Email address (different lengths)
- Age (whole numbers only)
- CGPA (numbers like 3.75, exact values needed)
- Is scholarship student? (Yes/No)
- Date of birth (dates only)
- Phone number (always 11 digits like "03001234567")

Your Task: Write the CREATE TABLE command with appropriate data types.

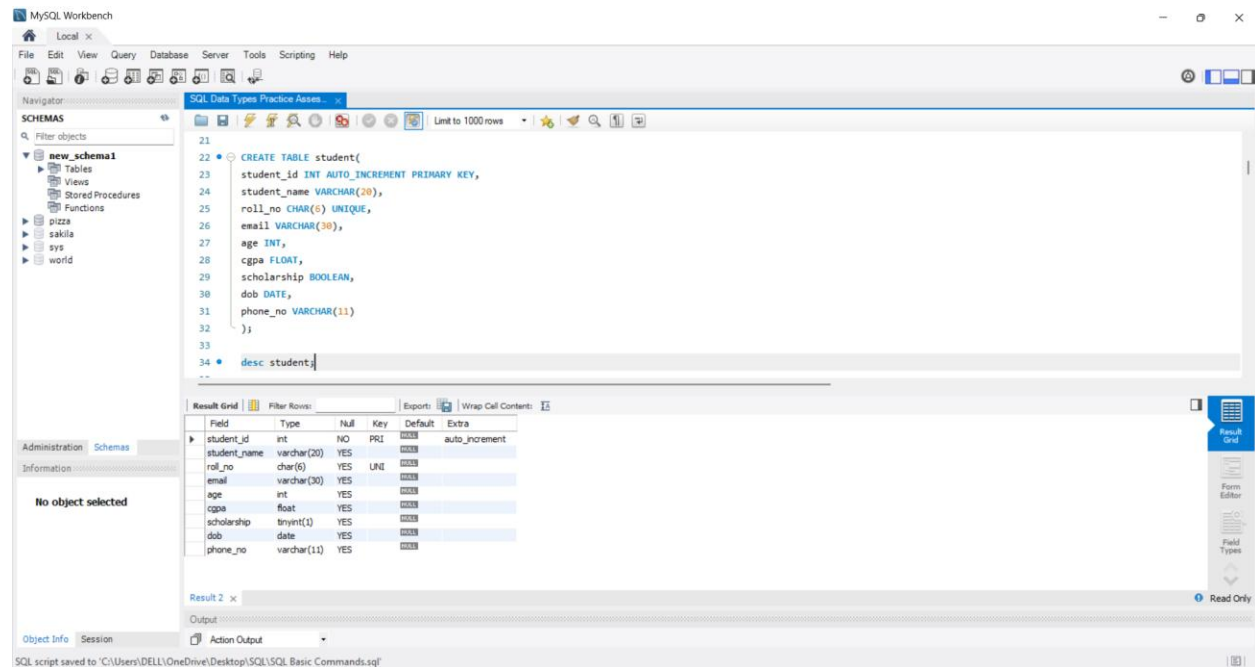
IMPLEMENTATION:

```
CREATE TABLE student(  
  student_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_name VARCHAR(20),  
  roll_no CHAR(6) UNIQUE,  
  email VARCHAR(30),  
  age INT,  
  cgpa FLOAT,  
  scholarship BOOLEAN,  
  dob DATE,
```

```
phone_no VARCHAR(11)
);
```

```
desc student;
```

OUTPUT:



Exercise 2: Library Books Table

Create a table called books for a library system.

What you need to store:

- Book ID (whole numbers, starts from 1)
- Book title (variable length)
- Author name (variable length up to 80 characters)
- ISBN (always 13 characters like "9781234567890")
- Category (must be one of: Fiction, Non-Fiction, Science, History)

- Price (exact money values like 599.99)
- Number of pages (whole numbers)
- Is available for lending? (Yes/No)
- Publication date (dates only)
- Brief description (long text)

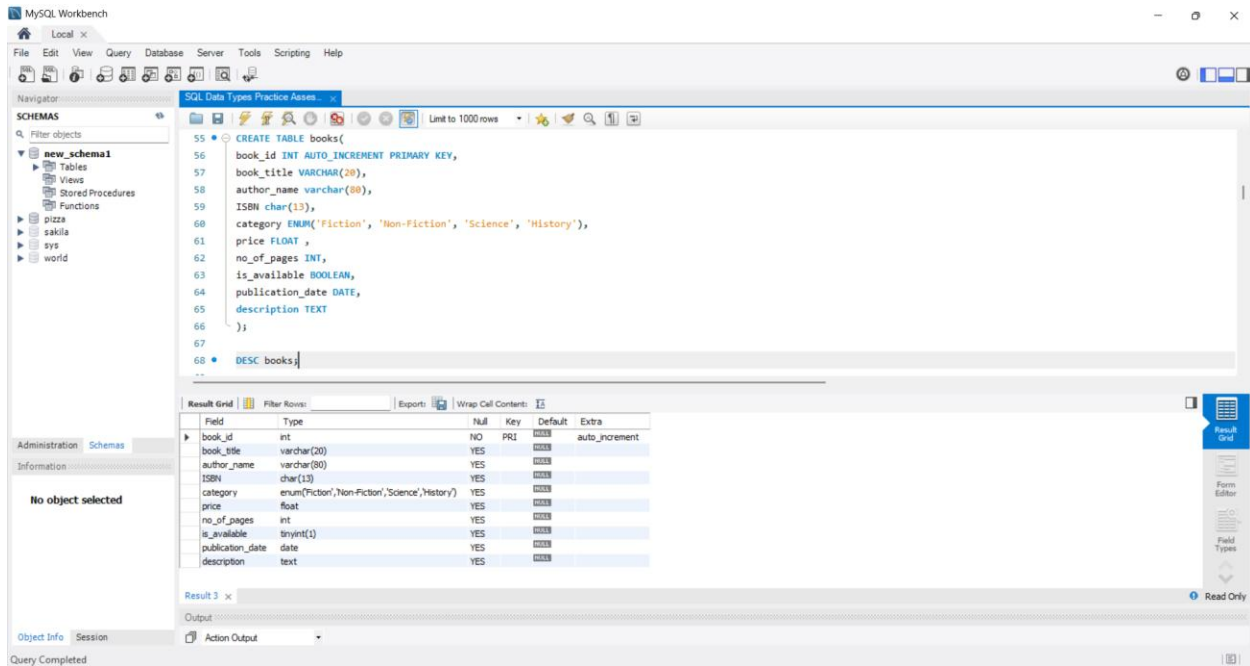
Your Task: Write the CREATE TABLE command.

IMPLEMENTATION:

```
CREATE TABLE books(
book_id INT AUTO_INCREMENT PRIMARY KEY,
book_title VARCHAR(20),
author_name varchar(80),
ISBN char(13),
category ENUM('Fiction', 'Non-Fiction', 'Science', 'History'),
price FLOAT ,
no_of_pages INT,
is_available BOOLEAN,
publication_date DATE,
description TEXT
);
```

```
DESC books;
```

OUTPUT:



Part 2: Understanding Different Number Types

Exercise 3: Online Store Orders

Create a table called orders that handles different types of numbers.

What you need to store:

- Order ID (very large numbers for millions of orders)
- Customer ID (regular whole numbers)
- Order amount (exact money - no rounding errors!)
- Tax amount (exact money)
- Shipping weight (approximate decimals like 2.567 kg)
- Number of items (small whole numbers 1-50)
- Is express delivery? (Yes/No)
- Order date and time together

- Discount percentage (approximate decimals)

Your Task:

1. Create the table with proper numeric data types
2. Insert these 3 sample orders:
 - Order 1: Customer 101, Amount 1250.75, Tax 125.08, Weight 3.2kg, 5 items, Express delivery, Today's date
 - Order 2: Customer 205, Amount 89.99, Tax 9.00, Weight 0.5kg, 1 item, Regular delivery, Yesterday
 - Order 3: Customer 156, Amount 2999.50, Tax 299.95, Weight 8.7kg, 12 items, Express delivery, Today

IMPLEMENTATION:

```
CREATE TABLE orders(
order_id BIGINT PRIMARY KEY AUTO_INCREMENT,
customer_id int,
order_amount DECIMAL,
tax_amount DECIMAL ,
shipping_weight FLOAT,
no_of_items TINYINT,
is_express BOOLEAN,
order_datetime DATETIME,
discount FLOAT,
CHECK (no_of_items BETWEEN 1 AND 50)
);
```

```
DESC orders;
```

```
INSERT INTO orders
```

```
(customer_id, order_amount, tax_amount, shipping_weight, no_of_items, is_express,
order_datetime, discount)
```

VALUES

(101, 1250.75, 125.08, 3.2, 5, True, '2026-03-29 10:15:00', 5.0),

(205, 89.99, 9.00, 0.5, 1, False, '2026-03-28 08:00:00', 5.0),

(156, 2999.50, 299.95, 8.7, 12, True, '2026-03-29 05:10:20', 5.0);

SELECT * FROM orders;

OUTPUT:

The first screenshot shows the MySQL Workbench interface with the 'CREATE TABLE orders' SQL statement entered in the query editor. The statement defines the table structure with columns: order_id (BIGINT, PRIMARY KEY, AUTO_INCREMENT), customer_id (int), order_amount (decimal), tax_amount (decimal), shipping_weight (float), no_of_items (TINYINT), is_express (BOOLEAN), order_datetime (DATETIME), discount (float), and a CHECK constraint on no_of_items between 1 and 50.

The second screenshot shows the same interface after inserting data into the 'orders' table. The SQL statement includes an 'INSERT INTO orders' statement with the following values:

```
VALUES
(101, 1250.75, 125.08, 3.2, 5, True, '2026-03-29 10:15:00', 5.0),
(205, 89.99, 9.00, 0.5, 1, False, '2026-03-28 08:00:00', 5.0),
(156, 2999.50, 299.95, 8.7, 12, True, '2026-03-29 05:10:20', 5.0);
```

The 'Result Grid' at the bottom of the second screenshot displays the data inserted into the 'orders' table:

order_id	customer_id	order_amount	tax_amount	shipping_weight	no_of_items	is_express	order_datetime	discount
1	101	1251	125	3.2	5	1	2026-03-29 10:15:00	5
2	205	90	9	0.5	1	0	2026-03-28 08:00:00	5
3	156	3000	300	8.7	12	1	2026-03-29 05:10:20	5

Questions to Answer:

1. Why did you choose DECIMAL for money instead of FLOAT?
➔ DECIMAL stores exact values, while FLOAT may cause rounding errors.
2. Why use BIGINT for Order ID?
➔ BIGINT can store very large numbers, suitable for millions/billions of orders.
3. What happens if you try to insert a negative number of items?
➔ If we try to insert a negative number of items, it will be inserted by default because the database does not automatically restrict it.

Part 3: Text and String Types

Exercise 4: User Registration System

Create a users table that shows the difference between CHAR and VARCHAR.

What you need to store:

- User ID (whole numbers)
- Username (3 to 25 characters)
- Country code (always 2 letters like "PK", "US")
- State code (always 3 letters like "SND", "PUN")
- Full name (variable length up to 100 characters)
- Password (always 32 characters - encrypted)
- Account type (Premium, Basic, Guest)
- About me (long biographical text)
- Verification code (always 6 digits like "123456")

Your Task:

1. Create the table using CHAR for fixed-length fields and VARCHAR for variable-length fields
2. Insert 5 different users with varying name lengths

IMPLEMENTATION:

```
CREATE TABLE users(  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  user_name VARCHAR(25),  
  country_code CHAR (2),  
  state_code CHAR(3),  
  full_name VARCHAR(100),  
  password VARCHAR(32),  
  account_type ENUM('Premium', 'Basic', 'Guest'),  
  about_me TEXT,  
  verification_code CHAR(6),  
  
  CHECK (CHAR_LENGTH(user_name) BETWEEN 3 AND 25)  
);
```

```
DESC users;
```

```
INSERT INTO users  
(user_name, country_code, state_code, full_name, password, account_type, about_me,  
verification_code)  
VALUES  
(  
'Ali', 'PK', 'PUN', 'Ali Khan', '12345678901234567890123456789012', 'Basic', 'Hello I am Ali',  
'123456'),  
  
(  
'Sara123', 'PK', 'SND', 'Sara Ahmed', 'abcdefghijklmnopqrstuvwxyzabcdefab', 'Premium', 'Love  
reading books', '654321'),
```

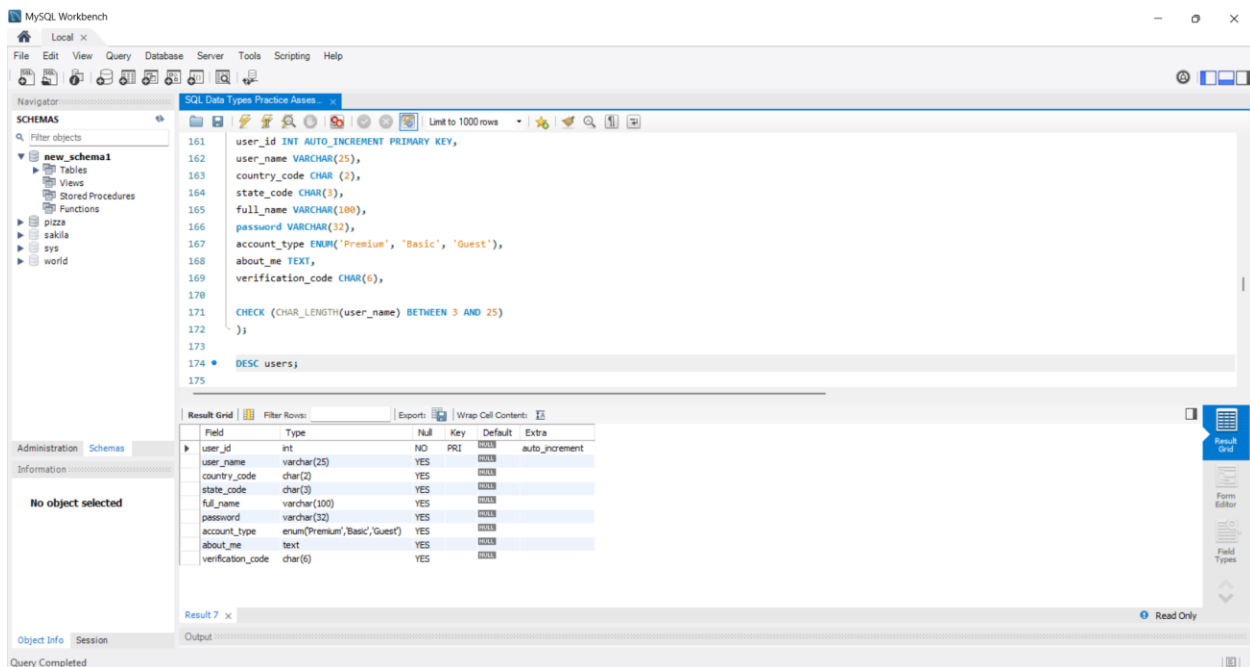

('JohnDoe', 'US', 'CAL', 'John Michael Doe', '11112222333344445555666677778888', 'Guest', 'Traveler and blogger', '112233'),

('AmanDeep', 'IN', 'PUN', 'Amandeep Kaur', '99998888777766665555444433332222', 'Premium', 'Student and developer', '445566'),

('MuhammadAbdullah', 'PK', 'KPK', 'Muhammad Abdullah Khan', 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaa', 'Basic', 'Learning SQL deeply', '778899');

SELECT * FROM users;

OUTPUT:



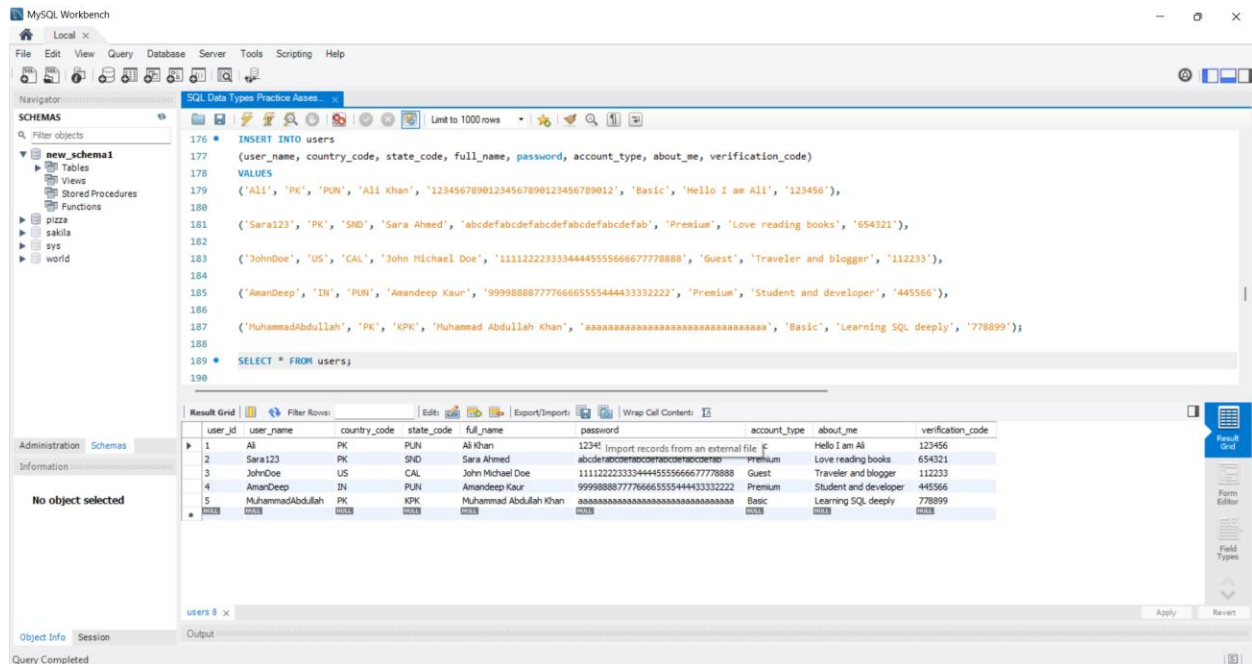
The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
161 user_id INT AUTO_INCREMENT PRIMARY KEY,  
162 user_name VARCHAR(25),  
163 country_code CHAR(2),  
164 state_code CHAR(3),  
165 full_name VARCHAR(100),  
166 password VARCHAR(32),  
167 account_type ENUM('Premium', 'Basic', 'Guest'),  
168 about_me TEXT,  
169 verification_code CHAR(6),  
170  
171 CHECK (CHAR_LENGTH(user_name) BETWEEN 3 AND 25)  
172 );  
173  
174 DESC users;  
175
```

The Result Grid shows the output of the query:

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI		auto_increment
user_name	varchar(25)	YES			
country_code	char(2)	YES			
state_code	char(3)	YES			
full_name	varchar(100)	YES			
password	varchar(32)	YES			
account_type	enum('Premium','Basic','Guest')	YES			
about_me	text	YES			
verification_code	char(6)	YES			

The bottom status bar indicates "Query Completed".



- Test Understanding: Insert a user with username "Ali" into your table, then insert another with username "MuhammadAbdullah". Explain the storage difference.
 - ➔ VARCHAR stores only actual data length, while CHAR uses fixed space, so shorter names waste space in CHAR but not in VARCHAR.
 - ➔ If username is VARCHAR(25), 'Ali' uses 3 characters and 'MuhammadAbdullah' uses 17 characters. Stores only actual length and save the space.
 - ➔ If username is CHAR(25), 'Ali' uses 25 characters (22 spaces added) and 'MuhammadAbdullah' uses 25 characters (8 spaces added) Always uses fixed length

Part 4: Dates and Times

Exercise 5: Event Management

Create an events table for managing school events.

What you need to store:

- Event ID (whole numbers)
- Event name (variable text)

- Event date (date only)
- Start time (time only)
- End time (time only)
- Registration deadline (date and time together)
- Created timestamp (automatically set when record is added)
- Is public event? (Yes/No)
- Max participants (small whole numbers)

Your Task:

1. Create the table
2. Insert 3 upcoming events with different dates and times
3. Write a query to find events happening today

IMPLEMENTATION:

```
CREATE TABLE events (
```

```
    event_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    event_name VARCHAR(100),
```

```
    event_date DATE,
```

```
    start_time TIME,
```

```
    end_time TIME,
```

```
    registration_deadline DATETIME,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    is_public BOOLEAN,
```

```
    max_participants TINYINT
```

```
);
```

```
DESC events;
```

```
INSERT INTO events
```

```
(event_name,    event_date,    start_time,    end_time,    registration_deadline,    is_public,
max_participants)
```

VALUES

('Science Fair', '2026-04-01', '10:00:00', '14:00:00', '2026-03-30 23:59:00', TRUE, 50),

('Sports Day', '2026-04-02', '08:00:00', '16:00:00', '2026-03-31 18:00:00', TRUE, 100),

('Coding Workshop', '2026-03-29', '11:00:00', '15:00:00', '2026-03-28 20:00:00', FALSE, 30);

SELECT * FROM events;

SELECT * FROM events

WHERE event_date = CURDATE();

OUTPUT:

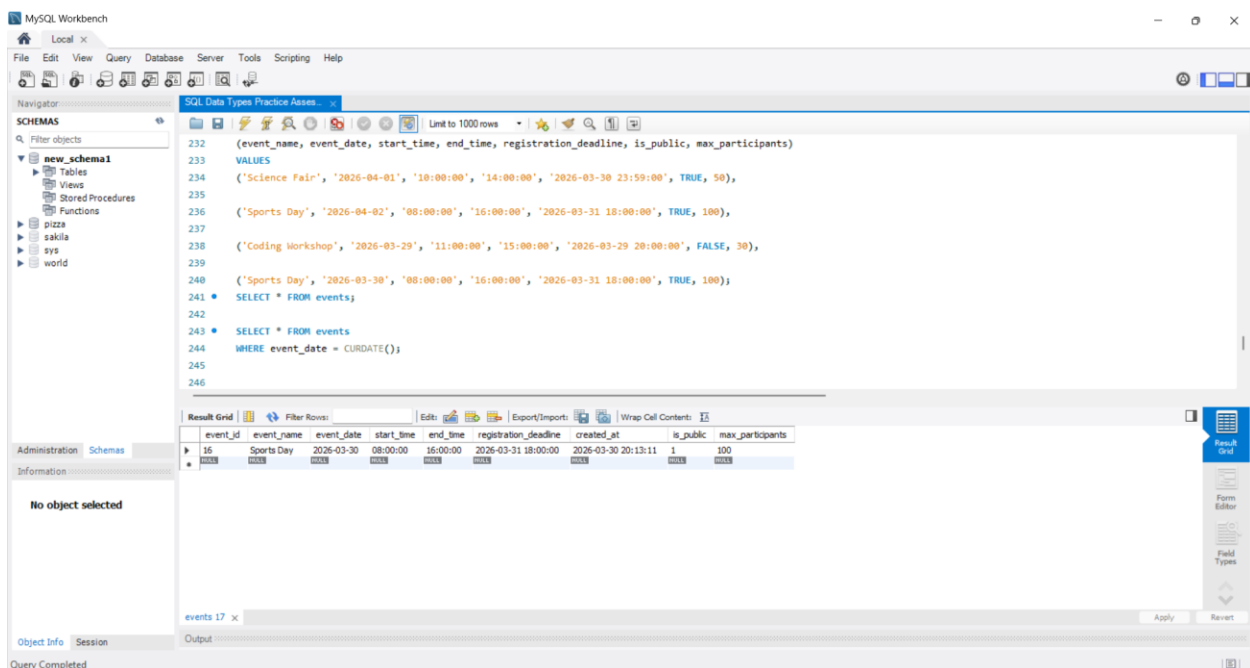
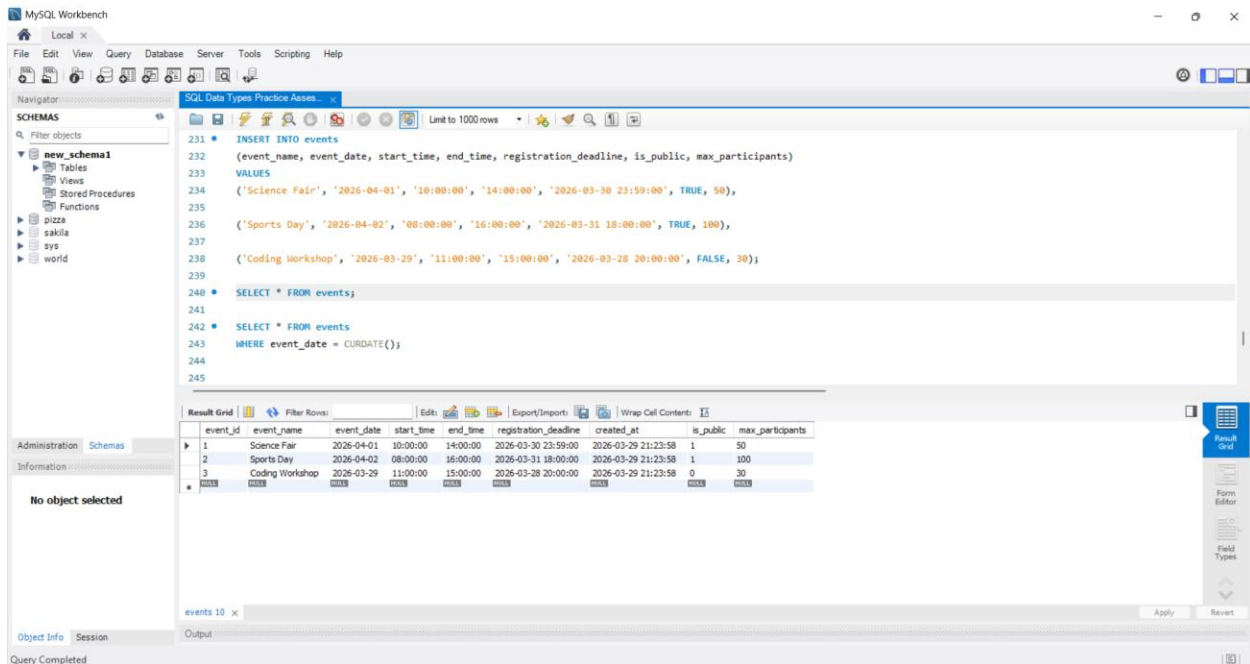
The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```
216  
217 CREATE TABLE events (  
218     event_id INT AUTO_INCREMENT PRIMARY KEY,  
219     event_name VARCHAR(100),  
220     event_date DATE,  
221     start_time TIME,  
222     end_time TIME,  
223     registration_deadline DATETIME,  
224     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
225     is_public BOOLEAN,  
226     max_participants TINYINT  
227 );  
228  
229 DESC events;  
230
```

The 'Result Grid' tab is active, displaying the table structure for 'events':

Field	Type	Null	Key	Default	Extra
event_id	int	NO	PRI	AUTO_INCREMENT	auto_increment
event_name	varchar(100)	YES			
event_date	date	YES			
start_time	time	YES			
end_time	time	YES			
registration_deadline	datetime	YES			
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
is_public	tinyint(1)	YES			
max_participants	tinyint	YES			

The left sidebar shows the 'SCHEMAS' panel with 'new_schema1' selected. The bottom status bar indicates 'Query Completed'.



Questions

1. When would you use CHAR instead of VARCHAR?
 ➔ CHAR is used when the data length is fixed and always the same. It allocates a fixed amount of storage, which makes it faster for retrieval.

- **Example:**
Country codes like IN, US → always 2 characters → use CHAR(2)
 - ➔ VARCHAR is better for variable-length data (like names).
2. Why is DECIMAL better than FLOAT for storing prices?
- ➔ DECIMAL stores exact numeric values, while FLOAT stores approximate values using binary representation, which can cause rounding errors.
 - **Example:**
 FLOAT: $0.1 + 0.2 = 0.30000000000000004$
 $0.1 + 0.2 = 0.30000000000000004$
 DECIMAL: $0.1 + 0.2 = 0.3$
 $0.1 + 0.2 = 0.3$
 - ➔ For money, accuracy is important, so DECIMAL is preferred.
3. What's the difference between DATE and DATETIME?
- ➔ DATE stores only the date part:
2026-03-29
 - ➔ DATETIME stores both date and time:
2026-03-29 14:30:00
 - ➔ Use DATE when time is not needed, and DATETIME when exact time is important (e.g., orders, events).
4. When should you use ENUM data type?
- ➔ ENUM is used when a column can have only a fixed set of predefined values.
 - **Example:**
status ENUM('Basic', 'Premium', 'Guest')
 - ➔ This ensures data consistency and prevents invalid entries.
5. What happens if you try to store text in an INT column?
- ➔ SQL will try to convert the text into a number.
 - If conversion is possible → it stores the number
 - If not → it stores 0 or gives an error (depending on SQL mode)
 - **Example:**
 INSERT INTO table_name (age) VALUES ('abc');
 Result: 0 (invalid data)